

Hantera felsituationer

Felsituationer händer ofta vid input från filer eller tangentbordet. Man kan aldrig vara säker på vad användaren skriver in eller vad som kan finnas i en fil som läses. Programmet måste helt enkelt vara förberett på att hantera fel av olika slag och kunna fortsätta exekveringen trots att inläst data inte på alla sätt uppfyller kriterierna för "korrekt data". Varje operation som vi använt tidigare i detta kapitel returnerar 0 ifall läsningen misslyckades, så vi kan använda oss av detta för att kontrollera att vi läser giltig data. Vi kan även använda oss av metoden `fail()` för att kontrollera om nästa läsning kommer att misslyckas. Problem får vi då vi skall bestämma oss för vad som skall göras med det felaktiga data som finns. Skall man läsa in och ignorera detta, och hur långt skall man i så fall ignorera input?

Vi kan skriva ett rudimentärt program som hanterar inläsning av heltal från tangentbordet på följande sätt:

```
#include <iostream>
#include <string>

int main () {
    int Tal;
    string Dummy;

    // iterera evigt
    while ( true ) {
        // läs ett tal
        cout << "Ge ett heltal: " << flush;

        // läs ett tal
        if ( ! (cin >> Tal) ) {
            // illegalt tal
            cout << "Illegalt heltal!" << endl;

            // nollställ felindikatorn för cin
            cin.clear ();

            // läs bort resten av raden och ignorera denna
            getline ( cin, Dummy );
        }
        else {
            // talet är ok
            cout << Tal << " är ett ok heltal!" << endl;
        }
    }
}
```

Notera parentesen run `cin >> Tal`. Utan denna kommer `!` att evalueras före `>>` tack vare högre precedens, och vi får ett resultat som inte är vad vi avser. Vi använder oss här av metoden `clear()` som nollställer alla felindikatorer hos `cin` för att vi skall kunna fortsätta läsa bort de illegala tecknen. Därefter läses alla tecken t.o.m. ett radbytestecken in i `Dummy` och ignoreras. På detta sätt får vi en rudimentär hantering av fel. Man nollställer alltså felflaggor med `clear()` enligt:

```
enFil.clear ();
```

Det finns även andra metoder som kan användas för `ifstream` och `istream`, nämligen `good()` och

`bad()`. Den förra anger att allting är i sin ordning och att nästa inläsning kan lyckas, medan `bad()` indikerar att någonting är allvarligt på tok och att inget kan sägas om vad som finns kvar att läsa. Skillnaden mellan `fail()` och `bad()` är att den förra indikerar ett fel i inläsningen men att inga tecken förlorats och man kan fortsätta läsa, medan den senare indikerar ett alvarligare fel som man kanske inte kan återhämta sig ifrån.

[Förutgående](#)

Skriva data till fil

[Hem](#)[Upp](#)[Nästa](#)

Stänga en fil